Kubernetes and Istio

Comprehensive Technical Documentation

Generated: June 2026
Version 1.0

7 - Volumes

In Kubernetes, a Pod is a logical computer that runs one or more containers. By default, each container has its own isolated, ephemeral filesystem. When a container restarts or crashes, any files written to its filesystem are permanently lost. To persist data across container restarts or to share files between containers in the same Pod, Kubernetes uses **Volumes**. A volume is a component defined at the Pod level that shares the Pod's lifecycle. It is created before any containers start and is destroyed only when the Pod is shut down.

Generic YAML declaration

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
spec:
  volumes:
  - name: my-storage-volume
    # ... volume type and configuration goes here ...
  containers:
  - name: my-container
    image: nginx
    volumeMounts:
    - name: my-storage-volume
      mountPath: /path/in/container
```

1. Available volume types

Kubernetes supports a wide variety of volume types tailored to different storage technologies and use cases.

Volume Type	Description
emptyDir	A simple empty directory created on the host node for the duration of the Pod's lifecycle. Best for temporary caching or sharing files between containers.
hostPath	Mounts a file or directory directly from the worker node's local filesystem into the Pod. Dangerous if used improperly, as it ties the Pod to a specific node and poses security risks.
nfs	Mounts a traditional Network File System share into the Pod.
gcePersistentDisk	Mounts a Google Compute Engine Persistent Disk. This is cloud-specific and requires the cluster to be running on GCP (other cloud providers use similar mechanism to provide storage).
cephfs	Mounts a CephFS network storage volume.
persistentVolumeClaim	A unique volume type that decouples the Pod from the underlying storage infrastructure, pointing instead to a persistent storage claim.

1.1 emptyDir

An `emptyDir` volume is the simplest way to provide a Pod with temporary storage that survives a container restart. Below is an example of a MongoDB Pod using an `emptyDir` to ensure its database files survive if the MongoDB container process crashes and restarts.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: quiz
spec:
  volumes:
  - name: quiz-data
    emptyDir: {}
  containers:
  - name: mongo
    image: mongo
    volumeMounts:
    - name: quiz-data
      mountPath: /data/db
```

emptyDir configuration options:
While often left empty (`{}`), an emptyDir volume supports specific configurations depending on performance requirements.

Field	Description
medium	Defines the storage medium. Leave blank to use the host node's default disk, or set to `Memory` to use `tmpfs` (a virtual memory filesystem) for extremely fast, RAM-based storage.
sizeLimit	Caps the total amount of local storage the directory can consume (e.g., 10Mi). Crucial for in-memory volumes to prevent memory exhaustion.

2. Mounting the volume in a container

Defining the volume in the Pod is only half the battle. You must use the volumeMounts array to map the volume into the container's file tree.

Field	Description
name	Must exactly match the name of a volume defined in `spec.volumes`.
mountPath	The destination path inside the container's filesystem.
readOnly	A boolean. If `true`, the container cannot modify the volume's contents.
subPath	Allows mounting a specific sub-directory of the volume instead of the entire root volume.

3. Shared volume between multiple containers

A single volume can be mounted into more than one container within the same Pod. This is a common pattern when using sidecar containers or init containers.

For instance, an init container might start first, run a script to generate static HTML files, and save them to an `emptyDir` volume. Once the init container terminates, a web server container (like Nginx) starts up, mounts that exact same volume at `/usr/share/nginx/html` , and serves the freshly generated files to users.

4. PersistentVolumes and PersistentVolumeClaim

Injecting direct storage configurations (like NFS IP addresses or GCE disk names) into a Pod manifest makes the application completely unportable. A developer moving their application from AWS to GCP would have to rewrite their Pod definitions.

To solve this, Kubernetes decouples infrastructure from application definitions using two objects: **PersistentVolume (PV)** and **PersistentVolumeClaim (PVC)**.

- **PersistentVolume (PV)**: Provisioned by the cluster administrator. It contains the low-level, infrastructure-specific details (e.g., GCE disk ID, NFS server IP).
- **PersistentVolumeClaim (PVC)**: Created by the application developer. It specifies the requirements for storage (e.g., "I need 1GiB of storage with read/write access").

Practical Example:
Instead of hardcoding a `gcePersistentDisk` into the Pod, the developer creates a PVC requesting 1Gi of space. Kubernetes binds this PVC to an available PV provided by the admin. The Pod manifest then simply references the PVC:

```yaml
volumes:
  - name: quiz-data
    persistentVolumeClaim:
      claimName: quiz-data-claim
```

The `PersistentVolumeClaim` is described as follows.

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: quiz-data-claim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

4.1 PersistentVolume access mode

When a Persistent Volume is created, it advertises how it can be accessed across the cluster's worker nodes.

Access Mode	Abbreviation	Description
ReadWriteOnce	RWO	The volume can be mounted in read/write mode by exactly one worker node at a time.
ReadOnlyMany	ROX	The volume can be mounted in read-only mode by multiple worker nodes simultaneously.
ReadWriteMany	RWX	The volume can be mounted in read/write mode by multiple worker nodes simultaneously.

4.2 Volume Access Patterns

When designing distributed applications, developers must carefully consider how Kubernetes handles concurrent storage access. Volume access modes (RWO, ROX, RWX) dictate concurrency limits at the **node level, not the pod level**, which heavily influences application architecture.

4.2.1 The ReadWriteOnce (RWO) Concurrency Misconception

A common misconception is that ReadWriteOnce means only a single Pod can write to the volume at a time. This is false. RWO means a single **worker node** can attach the volume in read/write mode.

- **Pod-Level Concurrency**: If multiple Pods are scheduled to the same node, they can all mount the RWO volume and write to it simultaneously.
- **Programming Best Practice**: Because Kubernetes simply provides the shared file tree to all Pods on that node, it does not manage file-level concurrency. If your architecture relies on multiple Pods writing to the exact same file concurrently, your application code must implement its own file-locking mechanisms to prevent data corruption.

4.2.2. Cross-Node Concurrency Blockers

If a volume is attached to Node A in read/write mode, Kubernetes will strictly block Node B from attaching it.

- **The `FailedAttachVolume` Error**: If you deploy a group of writer Pods across multiple nodes sharing a single RWO volume, only the Pods on the first node will run. Pods on other nodes will be permanently stuck in a ContainerCreating state with a `RESOURCE_IN_USE_BY_ANOTHER_RESOURCE` error.
- **Mixed Read/Write Blocking**: If a volume supports both RWO and ReadOnlyMany (ROX), and Node A mounts it for a writer Pod, Node B cannot mount it even for a read-only Pod. The read/write lock on Node A prevents any other node from accessing the volume until the writer Pod is terminated and the volume is detached.

4.2.3 Architectural Best Practice: Designing Distributed State

Because of these strict concurrency rules, developers must adopt specific architectural patterns when deploying stateful applications:

- **Avoid Shared Read/Write Volumes**: replicas of the same pod typically can’t use the same network volume in read/write mode. You should generally avoid designing applications that expect a shared, distributed file system for concurrent writes, unless you are specifically utilizing storage that natively supports ReadWriteMany (RWX), such as an NFS share.
- **One Volume Per Replica**: Instead of sharing a single PVC among multiple replicas, the best practice for stateful distributed applications (like MongoDB or Cassandra) is to ensure each Pod instance gets its own dedicated network storage volume. (Kubernetes automates this pattern using StatefulSets, a concept introduced in later chapters).
- **Decouple Writers from Readers**: If you must use a shared volume for a read-heavy application, ensure your writer Pods execute and terminate (e.g., using an Init Container or a Job) before spinning up your reader Pods. Once the writers release the volume, multiple nodes can concurrently attach the volume in ROX mode, allowing reader Pods to scale horizontally across the entire cluster without issue.

4.3 Dynamic Provisioning of Persistent Volumes

In large clusters, a system administrator cannot manually provision a `PersistentVolume` every time a developer requests a PVC. This bottleneck is solved via **Dynamic Provisioning**.

Using a **StorageClass** object, an administrator configures an automated volume provisioner (e.g., `kubernetes.io/gce-pd` for Google or `k8s.io/minikube-hostpath` for local testing).

When a developer submits a PVC, they specify the desired `storageClassName` . If a match is found, Kubernetes intercepts the request, dynamically reaches out to the cloud provider's API, provisions a new physical disk of the exact requested size, wraps it in a `PersistentVolume` object, and binds it to the developer's claim instantly—all without human intervention.